

**TITLE: EXP NO.: 7 ENDORSEMENT POLICY
CONFIGURATION IN HYPERLEDGER FABRIC**

1. AIM

To configure and implement endorsement policies in Hyperledger Fabric and verify how multiple organizations endorse transactions before they are committed to the blockchain ledger.

2. TOOLS / SOFTWARE REQUIRED

- Kali Linux / Ubuntu
- Docker
- Docker Compose
- Node.js (Version 18 or later)
- npm
- Git
- Hyperledger Fabric Samples
- Hyperledger Fabric Binaries
- Hyperledger Fabric Docker Images
- Visual Studio Code (Optional)

3. HARDWARE REQUIREMENTS

- Processor: Intel Core i3 or above
- RAM: Minimum 8 GB
- Storage: Minimum 20 GB Free Disk Space
- Internet Connection

4. THEORY

An Endorsement Policy defines which organizations must approve (endorse) a transaction before it can be committed to the ledger. Hyperledger Fabric uses endorsement policies to ensure that transactions are validated according to predefined business rules. For example: `OR('Org1MSP.peer','Org2MSP.peer')` means that either Org1 or Org2 can endorse the transaction. Similarly, `AND('Org1MSP.peer','Org2MSP.peer')` means that both organizations must endorse the transaction before it becomes valid. Endorsement

policies improve security, trust, and transaction integrity in permissioned blockchain networks.

5. PROCEDURE

Step 1: Navigate to the Test Network

```
cd ~/fabric-dev/fabric-samples/test-network
```

Step 2: Start the Fabric Network

```
./network.sh up createChannel -ca
```

Step 3: Deploy Chaincode with OR Endorsement Policy

Deploy the Asset Transfer chaincode with an endorsement policy that allows either organization to endorse transactions.

```
./network.sh deployCC  
-ccl javascript  
-ccn basic  
-ccp ../asset-transfer-basic/chaincode-javascript  
-ccep "OR('Org1MSP.peer','Org2MSP.peer')"
```

Step 4: Verify Chaincode Commitment

```
peer lifecycle chaincode querycommitted  
-C mychannel
```

Step 5: Initialize the Ledger

```
peer chaincode invoke  
-o localhost:7050  
--ordererTLSHostnameOverride orderer.example.com  
--tls  
--cafile $ORDERER_CA  
-C mychannel  
-n basic  
-c '{"function":"InitLedger","Args":[]}'
```

Step 6: Query All Assets

```
peer chaincode query  
-C mychannel  
-n basic  
-c '{"Args":["GetAllAssets"]}'
```

Step 7: Stop the Network

```
./network.sh down
```

Step 8: Restart the Network

```
./network.sh up createChannel -ca
```

Step 9: Deploy Chaincode with AND Endorsement Policy

Deploy the chaincode requiring endorsements from both organizations.

```
./network.sh deployCC
-ccl javascript
-ccn basic
-ccp ../asset-transfer-basic/chaincode-javascript
-ccep "AND('Org1MSP.peer','Org2MSP.peer')"
```

Step 10: Submit a Transaction

Invoke the chaincode transaction.

```
peer chaincode invoke
-o localhost:7050
--ordererTLSHostnameOverride orderer.example.com
--tls
--cafile $ORDERER_CA
-C mychannel
-n basic
-c '{"function":"CreateAsset","Args":["asset25","Blue","10","Arun","1200"]}'
```

Step 11: Verify the Asset

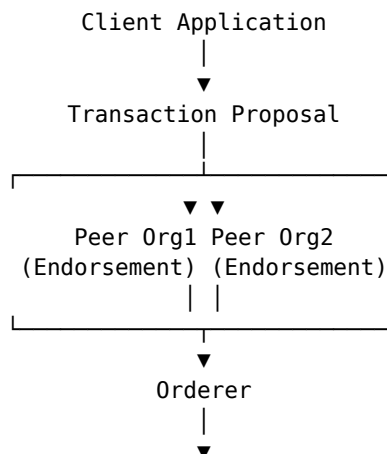
```
peer chaincode query
-C mychannel
-n basic
-c '{"Args":["ReadAsset","asset25"]}'
```

Step 12: Stop the Network

Flow Diagram Start Fabric Network | ▼ Deploy Chaincode | ▼ Configure Endorsement Policy | ▼ Submit Transaction Proposal | ▼ Collect Endorsements | ▼ Validate Transaction | ▼ Commit Transaction | ▼ Update Ledger | ▼ Query Ledger | ▼ Stop Network

```
./network.sh down
```

6. ARCHITECTURE DIAGRAM



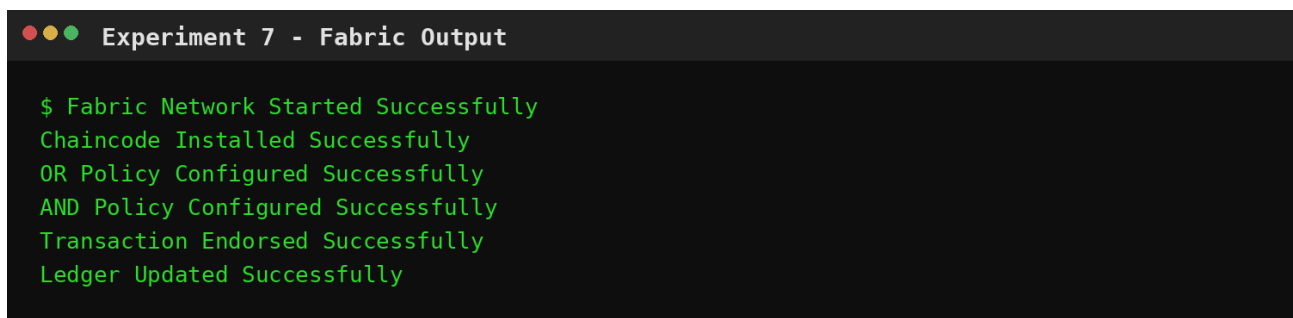
7. SUMMARY OF KEY COMMANDS

Purpose	Command
Navigate	<code>cd ~/fabric-dev/fabric-samples/test-network</code>
Start network	<code>./network.sh up createChannel -ca</code>
Deploy chaincode	<code>./network.sh deployCC</code>
Query ledger	<code>peer lifecycle chaincode querycommitted</code>
Invoke transaction	<code>peer chaincode invoke</code>
Query ledger	<code>peer chaincode query</code>
Stop network	<code>./network.sh down</code>

8. OUTPUT

The terminal output below is rendered from the output blocks supplied in the source experiment.

Output 1

A terminal window titled "Experiment 7 - Fabric Output" with a dark background and green text. The output shows a series of successful commands: "Fabric Network Started Successfully", "Chaincode Installed Successfully", "OR Policy Configured Successfully", "AND Policy Configured Successfully", "Transaction Endorsed Successfully", and "Ledger Updated Successfully".

```
Experiment 7 - Fabric Output

$ Fabric Network Started Successfully
Chaincode Installed Successfully
OR Policy Configured Successfully
AND Policy Configured Successfully
Transaction Endorsed Successfully
Ledger Updated Successfully
```

Fig. 7.1 – Fabric output corresponding to the source experiment output.

9. RESULT

The endorsement policies were successfully configured in the Hyperledger Fabric network. Transactions were validated according to the specified endorsement rules, demonstrating how Hyperledger Fabric ensures trust, integrity, and secure transaction processing through endorsement policies.